

Лабораторная работа 4-1. Сравнение подходов хранения больших данных

Цель работы: сравнить производительность и эффективность различных подходов к хранению и обработке больших данных на примере реляционной базы данных PostgreSQL и документо-ориентированной базы данных MongoDB.

Вариант 17

Задание для PostgreSQL

Социальная сеть. Создать таблицы users и posts (с user_id). Подсчитать количество постов для каждого пользователя, у которого более 10 постов.

1) Генерируем данные

```
from faker import Faker
import random
import uuid
from collections import Counter

fake = Faker()
random.seed(42)
Faker.seed(42)

num_users = 100_000
num_posts = 100_000
k_power_users = 10
min_posts_per_power = 15

users = [{"user_id": str(uuid.uuid4()), "name": fake.name()} for _ in range(num_users)]
user_ids = [u["user_id"] for u in users]
power_user_ids = random.sample(user_ids, k_power_users)

posts = []
for uid in power_user_ids:
    for _ in range(min_posts_per_power):
        posts.append({
            "post_id": str(uuid.uuid4()),
            "author_id": uid,
            "content": fake.text()
        })

remaining = max(0, num_posts - len(posts))
for _ in range(remaining):
    uid = random.choice(user_ids)
    posts.append({
        "post_id": str(uuid.uuid4()),
        "author_id": uid,
        "content": fake.text()
    })
```

```
counts = Counter(p["author_id"] for p in posts)
assert sum(1 for uid in power_user_ids if counts[uid] > 10) == k_power_users
```

- 2) Пишем скрипт для загрузки сгенерированных данных в БД + добавляем подсчитывание времени загрузки

Также считаем время выполнения данной по заданию агрегации

```
import psycopg2
from psycopg2.extras import execute_values
import json
import time

conn = psycopg2.connect(
    host="localhost", port="5433", database="studpg", user="pguser", password="pgpass"
)
cur = conn.cursor()

# Чистим и создаём JSONB-таблицы
cur.execute("DROP TABLE IF EXISTS users_json;")
cur.execute("DROP TABLE IF EXISTS posts_json;")
cur.execute("CREATE TABLE users_json (data JSONB);")
cur.execute("CREATE TABLE posts_json (data JSONB);")

#Загрузка JSON
start_load = time.time()

user_tuples = [(json.dumps(u),) for u in users] # <-- сохраняем как JSON
execute_values(cur, "INSERT INTO users_json (data) VALUES %s", user_tuples)

post_tuples = [(json.dumps(p),) for p in posts] # <-- сохраняем как JSON
execute_values(cur, "INSERT INTO posts_json (data) VALUES %s", post_tuples)

conn.commit()
time_postgres_load = time.time() - start_load

#Агрегация по JSON: авторы с >10 постов
query = """
SELECT
    data->>'author_id' AS author_id,
    COUNT(*)::int AS post_count
FROM posts_json
GROUP BY data->>'author_id'
HAVING COUNT(*) > 10
ORDER BY post_count DESC;
```

```

start_query = time.time()
cur.execute(query)
results_pg = cur.fetchall()
time_postgres_query = time.time() - start_query

cur.close()
conn.close()

print(f"PostgreSQL: загрузка {time_postgres_load:.2f} с, агрегация {time_postgres_query:.2f} с")
PostgreSQL: загрузка 3.25 с, агрегация 0.18 с

```

3) Проверяем успешность загрузки данных в БД и корректность выполнения агрегации

The screenshot shows the pgAdmin 4 web interface in a browser. The left sidebar displays the database structure, including a server named 'studpg' with a database 'public'. The main pane shows a SQL query executed: `SELECT * FROM posts_json;`. The 'Data Output' tab at the bottom displays the results of the query, showing 100,000 rows. The first few rows of the JSON data are visible:

data
1
2
3
4
5
6
7
8

At the bottom of the interface, it indicates 'Total rows: 100000' and 'Query complete 00:00:00.605'.

localhost:5050/browser/

pgAdmin

Object Explorer

- Servers (1)
 - 1
 - Базы данных (2)
 - postgres
 - Publications
 - Subscriptions
 - Каталоги
 - Обёртки сторонних данн
 - Приведения
 - Расширения
 - Событийные триггеры
 - Схемы
 - Языки
 - studpg
 - Publications
 - Subscriptions
 - Каталоги
 - Обёртки сторонних данн
 - Приведения
 - Расширения
 - Событийные триггеры
 - Схемы (1)
 - public
 - Aggregates
 - Operators
 - Анализаторы FTS
 - Домены
 - Конфигурации FTS
 - Материализованнь

studpg/pguser@1

Запрос История запросов

```
1 SELECT * FROM users_json;
```

Scratch Pad

Data Output Сообщения Notifications

Showing rows: 1 to 1000 Page No: 1 of 100

	data jsonb
1	{"name": "Allison Hill", "user_id": "40267527-c76d-420a-b3cb-e88bd523bd31"}
2	{"name": "Noah Rhodes", "user_id": "70b75225-e56e-4dc9-9f59-256afdd93545"}
3	{"name": "Angie Henderson", "user_id": "16f606d0-1f08-46e3-b3f8-7c310a371f0a"}
4	{"name": "Daniel Wagner", "user_id": "f1874cfc-8059-475d-a5d9-db6361039569"}
5	{"name": "Cristian Santos", "user_id": "fe0957f5-cffc-4144-9966-36eff366bf74"}
6	{"name": "Connie Lawrence", "user_id": "2ce38d7e-2add-450e-bdbf-885acb96b032"}
7	{"name": "Abigail Shaffer", "user_id": "e6c0ae83-2239-4eeb-a092-2688d40f5620"}
8	{"name": "Gina Moore", "user_id": "f30c7dd0-0feb-4342-97ab-a986c3e2ac11"}

Total rows: 100000 Query complete 00:00:00.304 LF Ln 1, Col 20

studpg/pguser@1

Запрос История запросов

```
1 SELECT
2     data->>'author_id' AS author_id,
3     COUNT(*)::int AS post_count
4 FROM posts_json
5 GROUP BY data->>'author_id'
6 HAVING COUNT(*) > 10
7 ORDER BY post_count DESC;
```

Data Output Сообщения Notifications

	author_id text	post_count integer
1	be414816-8123-4293-864e-ecd7ff443f05	18
2	f8742f0e-f96b-40e0-a563-2e367b5799e0	17
3	3cdb1659-1ef5-48e0-a1b7-bd579d8275...	17
4	deb94e38-23a3-47cb-8795-f9ad56101e...	16
5	5d38a1e9-d651-4437-8b36-b3d05d168...	15
6	6daf9c0a-e4fe-4a52-992c-2260be2a49...	15
7	f5320382-2a02-455a-bf5d-ba8711b68f...	15
8	ba519493-3fa5-4cef-b90c-71323190c68f	15
9	97743b82-073a-4cab-9404-2f9148d8d9...	15
10	f77e8d16-b205-4c52-9ba8-3317b85046...	15

Задание для MongoDB

Социальная сеть. Создать коллекцию posts (с author_id). Написать агрегацию для подсчета постов каждого автора и отфильтровать тех, у кого их более 10.

- 1) Так же сгенерированные данные загружаем в монго и по аналогии подсчитываем время загрузки и агрегации

```
from pymongo import MongoClient
import time

# Подключение
client = MongoClient("mongodb://mongouser:mongopass@localhost:27017/")
db = client.studmongo

# Очистка коллекций
db.users.delete_many({})
db.posts.delete_many({})

# ЗАГРУЗКА ДАННЫХ (меряем вместе users+posts)
start_load = time.time()

# users как есть
db.users.insert_many(users)

# posts: берём только нужные поля (author_id, content)
mongo_posts = [{"author_id": p["author_id"], "content": p["content"]} for p in posts]
db.posts.insert_many(mongo_posts)

# индексы (ускорят агрегацию)
db.users.create_index("user_id")
db.posts.create_index("author_id")

time_mongo_load = time.time() - start_load

# АГРЕГАЦИЯ (>10 постов на автора)
pipeline = [
    {"$group": {"_id": "$author_id", "post_count": {"$sum": 1}}},
    {"$match": {"post_count": {"$gt": 10}}},
    {"$project": {"_id": 0, "author_id": "$_id", "post_count": 1}},
    {"$sort": {"post_count": -1}}
]

start_query = time.time()
results_mongo = list(db.posts.aggregate(pipeline, allowDiskUse=True))
time_mongo_query = time.time() - start_query

client.close()

print(f"MongoDB: Время загрузки {time_mongo_load:.2f} с, Время агрегации {time_mongo_query:.2f} с")
MongoDB: Время загрузки 3.01 с, Время агрегации 0.28 с
```

- 2) Проверяем

Viewing Database: studmongo

Collections

Collection Name

+ Create collection

View

Export

[JSON]

Import

posts

Del

View

Export

[JSON]

Import

users

Del

Database Stats

Collections (incl. system.namespaces)	2
Data Size	33.1 MB
Storage Size	23.4 MB
Avg Obj Size #	166 Bytes
Objects #	200000
Indexes #	5
Index Size	10.6 MB

Viewing Collection: posts

New Document New Index

Simple

Advanced

Key

Value

String

Find

Delete all 100000 documents retrieved

1 2 3 4 5 > >>

_id	author_id	content
6915da21ea5d8072ea733aca	c25dee76-5bf4-43d9-adcf-b4be7b34840a	Window all detail pressure mention. Arrive far so...
6915da21ea5d8072ea733acb	c25dee76-5bf4-43d9-adcf-b4be7b34840a	Or herself according man. Memory task church poli...
6915da21ea5d8072ea733acc	c25dee76-5bf4-43d9-adcf-b4be7b34840a	Character individual nearly alone discussion far ...
6915da21ea5d8072ea733acd	c25dee76-5bf4-43d9-adcf-b4be7b34840a	Want performance audience though. Forget why only...
6915da21ea5d8072ea733ace	c25dee76-5bf4-43d9-adcf-b4be7b34840a	Us major this. Open nor value score finish offici...

localhost:8081/db/studmongo/users

Mongo Express Database: studmongo Collection: users

Viewing Collection: users

New Document New Index

Simple Advanced

Key Value String Find

Delete all 100000 documents retrieved

1 2 3 4 5 > >>

_id	user_id	name
6915da20ea5d8072ea71b42a	c1e2d461-b4b4-4d82-a1f8-b08f6c223984	Allison Hill
6915da20ea5d8072ea71b42b	11a13926-4e77-44ae-9f89-27a08121a4f7	Noah Rhodes
6915da20ea5d8072ea71b42c	7d7fb1a0-2e46-4b6f-83c0-a2cfb3f30cb5	Angie Henderson
6915da20ea5d8072ea71b42d	6338e054-f433-4404-969e-f09510f1ddcd	Daniel Wagner
6915da20ea5d8072ea71b42e	cc7b0e60-1a57-4702-a0c1-48cfcd06fcd4	Cristian Santos

localhost:8081/db/studmongo/posts?query=%5B%0D%A++++%7B"%24group"%3A+%7B"_id"%3A+%24author_id"%2C+"po...

Mongo Express Database: studmongo Collection: posts

```
[{"$group": {"_id": "$author_id", "post_count": {"$sum": 1}}], [{"$match": {"post_count": {"$gt": 10}}], [{"$project": {"_id": 0, "author_id": "$_id", "post_count": 1}}, {"$sort": {"post_count": -1}}]
```

Projection

☒ Aggregate query

Find

Delete all 10 documents retrieved

post_count	author_id
18	72ff01f0-ae9b-49b2-b505-54e199cf7a63
17	dd4eeb0f-01fa-40d4-bc74-8dbf48a20770
17	a115d1a4-2610-44d5-baee-c9a52c05a512
16	80b7e8f4-699d-4bf2-bc86-d62c105b2f7c
15	2fffd5a6-6511-4b1f-98cb-e3ad3ad671e3
15	59b07aaf-840a-42c7-8e9c-eee12cf925b3
15	f5e767f3-8dc0-4bff-bdb4-327e7afb54d
15	f979b124-9e86-43bf-b850-66e061e14165
15	332773b5-2895-492a-a5a9-34f3d10ddd60
15	e6e01233-7c6a-45b3-a738-aed86db2c334

Анализ в Jupyter Notebook

1) Собираем данные в общую таблицу


```

import pandas as pd

# Создаём словарь с данными
data = {
    "База данных": ["PostgreSQL", "MongoDB"],
    "Время загрузки (с)": [time_postgres_load, time_mongo_load],
    "Время агрегации (с)": [time_postgres_query, time_mongo_query]
}

# Создаём DataFrame
df = pd.DataFrame(data)

# добавим столбец с общим временем
df["Общее время (с)"] = df["Время загрузки (с)"] + df["Время агрегации (с)"]

# Выводим таблицу
print(df)

```

	База данных	Время загрузки (с)	Время агрегации (с)	Общее время (с)
0	PostgreSQL	3.247826	0.184420	3.432246
1	MongoDB	3.011166	0.280675	3.291841

2) Строим график разницы во времени запросов по двум БД

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Данные (секунды)
data = {
    "База данных": ["PostgreSQL", "MongoDB"],
    "Время загрузки": [time_postgres_load, time_mongo_load],
    "Время агрегации": [time_postgres_query, time_mongo_query]
}
df = pd.DataFrame(data)

fig, ax = plt.subplots(figsize=(8, 5))
x = np.arange(len(df))
w = 0.5

# Цвета: PG – синий, Mongo – зелёный (кап – темнее)
pg_base, pg_cap = "#5aa5df", "#1f77b4"
mg_base, mg_cap = "#6bd36b", "#2ca02c"

# База = загрузка; "кап" = агрегация внутри столбика
for i, (base_col, cap_col) in enumerate([(pg_base, pg_cap), (mg_base, mg_cap)]):
    load = float(df.loc[i, "Время загрузки"])
    agg = float(df.loc[i, "Время агрегации"])

    ax.bar(x[i], load, w, color=base_col, edgecolor="none")
    ax.bar(x[i], agg, w,
           bottom=load - agg, color=cap_col,
           edgecolor=cap_col, linewidth=1.0, zorder=3,
           label=("PostgreSQL – агрегация" if i == 0 else "MongoDB – агрегация"))

# Оформление
ax.set_title("Время загрузки с выделением сегмента агрегации (внутри столбика)")
ax.set_xlabel("База данных")
ax.set_ylabel("Время (секунды)")
ax.set_xticks(x)
ax.set_xticklabels(df["База данных"])

```



```

# Начинаем ось Y с 2.5 сек
lower = 2.5
upper = max(float(df["Время загрузки"].max()) * 1.05, lower * 1.10)
ax.set_ylim(lower, upper)

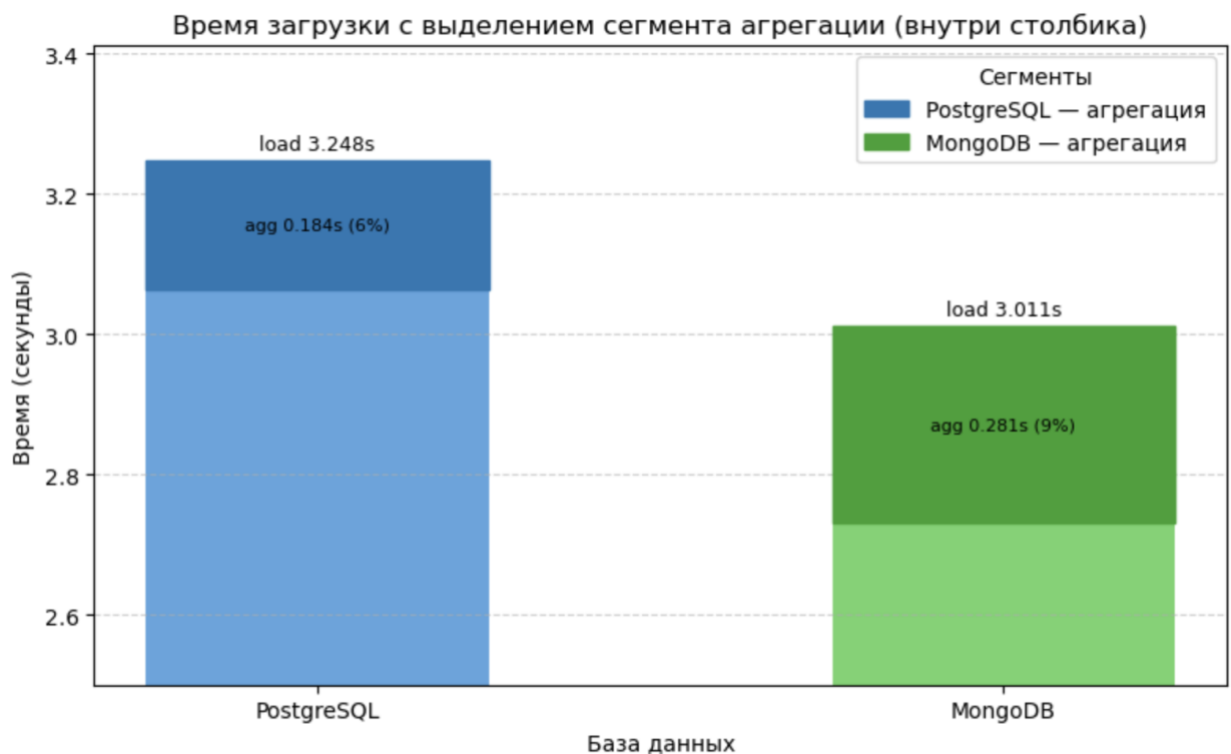
# Аннотации
for i in range(len(df)):
    load = df.loc[i, "Время загрузки"]
    agg = df.loc[i, "Время агрегации"]
    ax.annotate(f"load {load:.3f}s", (x[i], load),
               ha="center", va="bottom", fontsize=9, xytext=(0, 3),
               textcoords="offset points")
    ax.annotate(f"agg {agg:.3f}s ({agg/load*100:.0f}%)",
               (x[i], load - agg/2),
               ha="center", va="center", fontsize=8)

ax.yaxis.grid(True, linestyle="--", alpha=0.6)

# Легенда: берём уникальные метки от нарисованных объектов
handles, labels = ax.get_legend_handles_labels()
uniq = dict(zip(labels, handles))
ax.legend(uniq.values(), uniq.keys(), title="Сегменты")

plt.tight_layout()
plt.show()

```



Вывод

При больших объемах данных монго быстрее справляется с их загрузкой, скорость агрегации примерно сопоставима у обеих БД, но немного быстрее у постгре.